NLP & Tokenization

From Raw Text to Transformer-Ready Tensors

1

Where We Are

- ✅ Previously: The **original Transformer** (Vaswani et al., 2017) — attention, FFN, residuals, layer norm, sinusoidal PE
- ❓ Today: **NLP fundamentals** — tokenization, embeddings, the output pipeline, loss functions, and metrics
- 🔜 Next: **Modern transformer upgrades** — RoPE, RMSNorm, SwiGLU, GQA, KV-cache, flash attention, etc.
- 🔜🔜 Then: **GPT-OSS** — tokenizer training, data pipeline, model implementation, pre-training loop
- Today's goal: understand **every piece of the text ↔ tensor pipeline** so nothing is a black box when we start learning GPT-OSS

2

Part 1: The Text-to-Tensor Pipeline

3

The Complete Pipeline at a Glance

- Transformers operate on **tensors** — text is **discrete symbols** — we need a bridge in both directions
- **Stage 1 — Tokenization**: Raw text → sequence of subword tokens → integer IDs
- **Stage 2 — Embedding**: Token IDs → dense vectors via learned lookup table
- **Stage 3 — Positional info**: Inject sequence order (sinusoidal / learned / RoPE)
- **Stage 4 — Transformer layers**: Contextualize embeddings via attention + FFN
- **Stage 5 — LM Head**: Project back to vocabulary size → logits (raw scores)
- **Stage 6 — Loss** (training): Compare logits to ground truth → cross-entropy → backpropagate
- **Stage 7 — Generation** (inference): Logits → softmax → sampling → decoded text

4

Part 2: Tokenization

5

What Is a Token?

- A **token** is the atomic unit the model operates on — each token gets one integer ID and one embedding vector
- Three possible granularities, each with fundamental trade-offs:
- **Character-level**: "Hello" → ["H","e","l","l","o"] — tiny vocab (~256) but sequences 4-5× longer → far more compute
- **Word-level**: "Hello" → ["Hello"] — intuitive but vocab explodes (500K+ words), unseen words cause hard failures
- **Subword-level**: "unhappiness" → ["un","happi","ness"] — manageable vocab (32K–200K), handles any word, captures morphology
- **Most modern LLMs use subword tokenization** — GPT, Claude, Gemini, DeepSeek, Qwen

6

Why Subwords Win

- Consider: "play", "playing", "played", "plays", "player", "playful", "replaying"
- **Word-level**: 7 completely separate tokens — model must independently learn each one's meaning
- **Subword**: All share the token "play" — the model inherently learns they are related
- **OOV problem**: A word-level tokenizer can't handle novel words like "GPT-OSS" — it's unknown
- A subword tokenizer decomposes it: ["GPT", "-", "OSS"] — always produces something meaningful
- **Key trade-off**: Smaller vocab → longer sequences → more compute; Larger vocab → bigger embedding table → more parameters
- Modern trend: vocabularies are growing (GPT-5.2: ~200K, Qwen3: ~150K) because compute is cheaper than context length

7

Byte-Pair Encoding (BPE) — The Algorithm

- BPE is the dominant tokenization algorithm in modern LLMs
- Originally a data compression algorithm (Gage 1994); adapted for NLP (Sennrich et al. 2016)
- **The tokenizer is trained separately, on your text corpus, before model training begins:**
- 1. Start with a base vocabulary of individual units (characters or bytes)
- 2. Scan the corpus and count all adjacent token pairs
- 3. Merge the most frequent pair into a single new token; add it to the vocabulary
- 4. Repeat steps 2–3 until the vocabulary reaches your target size (e.g., 100K–200K)
- The output is an ordered **merge table**
- The merge table + base vocabulary + normalization + pre-tokenization rules define the tokenizer

8

BPE: Worked Example

- Training corpus: "low low low low lower lower newest newest newest widest"
- **Initial vocabulary**: All individual characters — {l, o, w, e, r, n, s, t, i, d, ...}
- **Iteration 1**: Most frequent pair (e, s) appears 4× → merge to "es"
- **Iteration 2**: Most frequent pair (es, t) appears 4× → merge to "est"
- **Iteration 3**: Most frequent pair (l, o) appears 6× → merge to "lo"
- **Iteration 4**: Most frequent pair (lo, w) appears 6× → merge to "low"
- Continue until target vocab size is reached
- **At inference**: Apply the same merge rules in the same order to tokenize any new text

🚀 **Interactive Demo:** bpe_algorithm_demo.html

9

Byte-Level BPE — What LLMs Actually Use

- Standard BPE starts from Unicode characters — but Unicode has **150,000+** code points
- **Byte-level BPE** starts from the **256 raw bytes** instead — guaranteeing a tiny, universal base vocab
- Process: UTF-8 encode the text into bytes first, then run BPE merges on the byte sequence
- **Result**: Any text in any language, any emoji, any code snippet can be represented — **no unknown tokens, ever**
- Introduced by GPT-2; now used by many frontier models
- **Vocabulary composition**: 256 byte tokens + N BPE merges + special tokens = total vocab
- Example: GPT-2 = 50,257 tokens; GPT-OSS and modern frontier models = 100K–200K tokens

10

Vocabulary Size — A Critical Hyperparameter

- Vocab size is fixed **before training** and cannot be changed afterward:
- • GPT-2 (2019): **50,257** tokens
- • DeepSeek V3.2: **~128,000** tokens
- • Qwen3: **~152,000** tokens
- • GPT-5.2 / GPT-OSS: **~200,000** tokens
- **Larger vocab** → shorter sequences (fewer tokens per text) → faster training and inference, but bigger embedding table
- **Smaller vocab** → longer sequences → higher compute cost per example
- **Modern trend**: larger vocabularies — especially important for multilingual and code-heavy training data
- For GPT-OSS, **choosing vocab size is one of the very first design decisions** you'll make

11

Special Tokens

- Beyond text tokens, models need **structural markers** added to the vocabulary:
- **<|endoftext|>** or **<|eos|>**: End of sequence / document boundary — model learns to stop generating here
- **<|begin_of_text|>** or **<|bos|>**: Beginning of sequence marker
- **<|pad|>**: Padding token — fills shorter sequences in a batch to equal length
- **Chat tokens**: <|im_start|>, <|im_end|>, <|user|>, <|assistant|> — structure multi-turn conversations
- These are **manually added** after BPE training — each gets its own learned embedding

12

Tokenizer Implementations

- **tiktoken** (OpenAI): Rust-based, extremely fast BPE — used by GPT-OSS
- **SentencePiece** (Google): Language-agnostic, supports BPE and Unigram — used historically by Google models
- **Unigram** (Kudo 2018): Starts with large vocab, iteratively removes low-impact tokens — used inside SentencePiece
- **WordPiece**: Merges by likelihood gain rather than frequency — older models only (not used in modern LLMs)
- **HuggingFace Tokenizers**: Fast Rust-backed library wrapping multiple algorithms — what we'll use in practice
- Key insight: the tokenizer is trained **separately** from the model, **before** model training begins — it never changes afterward

13

Tokenization Artifacts — Why This Matters for Model Behavior

- Tokenization directly causes many surprising LLM failures:
- **Arithmetic**: "128" = 1 token, "129" = ["12","9"] — completely different structures for consecutive numbers
- **Spelling**: "What letters are in 'ghost'?" — model sees ["ghost"], never individual letters
- **Whitespace**: " hello" and "hello" are different tokens with different embeddings
- **Multilingual inequality**: English ≈ 1 token/word; Chinese/Japanese ≈ 2–4 tokens/word for the same meaning → 2–4× more compute
- **Code formatting**: Indentation (tabs vs spaces) can wildly change token counts
- **Tokenizer training quality directly affects model quality** — this is not a trivial preprocessing step

🚀 **Interactive Demo:** tokenizer_demo.html

14

Working with Tokenizers in Practice

PYTHON

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "Tokenization is surprisingly important!"

# See the subword tokens (strings)
tokens = tokenizer.tokenize(text)
print(tokens)  # ['Token', 'ization', 'Ġis', 'Ġsurprisingly', 'Ġimp
# Ġ = leading space — byte-level BPE encodes spaces as part of toke

# Encode: text → integer IDs
ids = tokenizer.encode(text)
print(ids)  # [30642, 1634, 318, 11242, 1593, 0]

# Full tokenization (returns tensors + attention mask for batching)
inputs = tokenizer(text, return_tensors="pt", padding=True)
print(inputs.input_ids)       # Token IDs as tensor
print(inputs.attention_mask)  # 1 = real token, 0 = padding

# Decode: IDs → text (lossless roundtrip)
print(tokenizer.decode(ids))  # "Tokenization is surprisingly impor

print(f"Vocabulary size: {len(tokenizer)}")  # 50257
```

Exploring Tokenization Artifacts

PYTHON

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")

# Numbers are tokenized inconsistently
for n in ["127", "128", "129", "1000", "10000"]:
    toks = tokenizer.tokenize(n)
    print(f"{n:>6} → {toks}  ({len(toks)} tokens)")
# 127   → ['127']          (1 token)
# 128   → ['128']          (1 token)
# 129   → ['12', '9']      (2 tokens!)  ← different structure
# 1000  → ['1000']         (1 token)
# 10000 → ['100', '00']    (2 tokens)

# Multilingual cost inequality
en = "The cat sat on the mat."
zh = "猫坐在垫子上。"  # Same meaning in Chinese
print(f"English: {len(tokenizer.encode(en))} tokens")
print(f"Chinese: {len(tokenizer.encode(zh))} tokens")  # 2-3× more!

# These artifacts explain many real model failures
```

16

Part 3: Embeddings — From IDs to Vectors

17

Token Embeddings

- Token IDs are just integers — ID 15496 has no mathematical relationship to ID 15497
- The model needs **dense vectors** where distance and direction encode meaning
- **Embedding layer**: A learned lookup table — a matrix W_E of shape (vocab_size × embed_dim)
- Token ID → row index → retrieve that row = the token's embedding vector
- Example: ID 15496 → [0.12, −0.34, 0.87, ...] ∈ ℝ^4096
- Vectors are **randomly initialized** and then **learned during pre-training** via backpropagation
- After training, tokens used in similar contexts end up with similar vectors — semantic structure emerges

18

The Embedding Matrix (W_E)

- This is one of the model's **largest parameter groups**:
- • GPT-2: 50,257 × 768 = **38.6M parameters** in embeddings alone
- • GPT-OSS scale: 200K × 4096 = **~819M parameters** — nearly 1B just for the lookup table!
- Each row is the learned representation of one token
- After training, the space captures analogies: vector("king") − vector("man") + vector("woman") ≈ vector("queen")
- **Weight tying**: Many modern LLMs reuse W_E as the output projection layer (LM head) — same matrix, transposed
- This saves hundreds of millions of parameters and enforces consistency between input and output representations

🚀 **Interactive Demo:** embeddings_demo.html

19

Positional Information

- Recall from the transformer lecture: **self-attention is permutation-invariant**
- Without position info, "dog bites man" = "man bites dog" — identical outputs!
- The model needs to know **where** each token sits in the sequence
- **Original Transformer (2017)**: Fixed sinusoidal positional encodings — mathematical formula, not learned
- **GPT-2 (2019)**: Learned positional embeddings — a second lookup table of shape (max_seq_len × embed_dim)
- Input to first transformer layer = token_embedding + positional_embedding (element-wise addition)
- **Modern models (DeepSeek V3.2, Qwen3, GPT-OSS)**: Use **RoPE** (Rotary Position Embeddings) instead
- RoPE encodes relative position via rotation matrices applied inside attention

20

Static vs Contextual Representations

- The embedding lookup gives each token a **static** vector — the same vector regardless of context
- But language is deeply contextual: "bank" means different things in "river bank" vs "bank account"
- **This is exactly what the transformer layers solve:**
- Token enters layer 1 with its static embedding → attention mixes information from all visible positions → FFN transforms → repeat across N layers
- After all layers, each position holds a **contextual representation** — shaped by the entire sequence
- Older approaches (Word2Vec, GloVe, 2013–2017) only had static embeddings — one fixed vector per word, forever
- The transformer architecture you already learned is what makes contextual representations possible

21

The Input Pipeline in PyTorch

PYTHON

```
import torch
import torch.nn as nn

class InputPipeline(nn.Module):
    """Token + positional embeddings for a GPT-style model.
    GPT-OSS replaces pos_embed with RoPE."""
    def __init__(self, vocab_size=200000, max_seq_len=4096, embed_d
        super().__init__()
        self.token_embed = nn.Embedding(vocab_size, embed_dim)
        self.pos_embed = nn.Embedding(max_seq_len, embed_dim)  # →
        self.dropout = nn.Dropout(0.1)

    def forward(self, token_ids):  # (batch_size, seq_len)
        B, T = token_ids.shape
        tok_emb = self.token_embed(token_ids)                  # (
        positions = torch.arange(T, device=token_ids.device)    # [
        pos_emb = self.pos_embed(positions)                    # (
        x = self.dropout(tok_emb + pos_emb)                    # (
        return x  # → feeds into the first transformer layer

# Parameter count
pipeline = InputPipeline()
print(f"Embedding parameters: {sum(p.numel() for p in pipeline.para
# ~835M parameters just for embeddings + positions!
```

22

Part 4: The Output Pipeline — From Vectors to Text

23

The Language Modeling Head

- The transformer outputs one vector per position: shape **(batch, seq_len, embed_dim)**
- For next-token prediction, we need a **score for every token in the vocabulary**
- **LM Head**: A linear projection from embed_dim → vocab_size (no bias)
- Output shape: **(batch, seq_len, vocab_size)** — one score per vocab entry per position
- These raw scores are called **logits** — they can be any real number
- **With weight tying**: $logits = hidden @ W_E^T$ — multiply by the transposed embedding matrix
- At position t, the logits represent the model's prediction for what token comes at position t+1

24

Logits → Probabilities

- **Logits** are raw scores — not interpretable as probabilities
- e.g., [2.1, −0.5, 8.3, 0.1, ...] — one value per vocab entry (~200K values)
- Apply **softmax** to convert to a valid probability distribution:
- $P(token_i) = \frac{exp(logit_i)}{\Sigma_j exp(logit_j)}$ — all values ∈ (0,1), sum to 1.0
- Higher logit → higher probability → model thinks this token is more likely
- Example: logits [2.1, 5.8, 0.3, ...] → probabilities [0.02, 0.89, 0.001, ...]
- During **training**: compare this distribution to the true next token via cross-entropy loss
- During **inference**: select a token from this distribution via a decoding strategy

25

Decoding Strategies (Inference Time)

- Given probabilities over ~200K tokens, how do we pick the next one?
- **Greedy decoding**: Always pick argmax — deterministic but often repetitive and boring
- **Temperature scaling**: Divide logits by T before softmax — $T < 1$ sharpens (more confident), $T > 1$ flattens (more creative), $T \to 0$ = greedy
- **Top-k sampling**: Keep only the k most probable tokens, zero out the rest, renormalize, then sample
- **Top-p (nucleus) sampling**: Keep the smallest set of tokens whose cumulative probability $\geq p$ (e.g., 0.95), then sample
- **Modern LLM defaults**: Combine temperature + top-p — e.g., temperature=0.7, top_p=0.95
- During **training**, we don't sample at all — we compute loss at every position simultaneously

26

Autoregressive Generation Loop

- GPT-style models generate text **one token at a time** (autoregressive):
- 1. Input: "The cat" → forward pass → logits at last position → sample → "sat"
- 2. Input: "The cat sat" → forward pass → logits at last position → sample → "on"
- 3. Input: "The cat sat on" → forward pass → sample → "the"
- 4. Repeat until <|endoftext|> token or max length is reached
- Each step requires a forward pass — **this is fundamentally why generation is slow**
- **KV-cache**: An optimization that caches previous key/value computations to avoid redundant work — covered next lecture
- The entire architecture of GPT-OSS is designed to make this loop work well

27

The Output Pipeline in PyTorch

PYTHON

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# After transformer layers, we have hidden states
# hidden: (batch, seq_len, embed_dim)

# --- LM HEAD: Project to vocabulary ---
lm_head = nn.Linear(embed_dim, vocab_size, bias=False)
logits = lm_head(hidden)  # (batch, seq_len, vocab_size)

# --- TRAINING: Compute loss ---
# Position t's logits predict token at position t+1, so we shift:
shift_logits = logits[:, :-1, :].contiguous()   # (batch, T-1, voca
shift_labels = token_ids[:, 1:].contiguous()     # (batch, T-1)
loss = F.cross_entropy(
    shift_logits.view(-1, vocab_size),  # Flatten to (batch*(T-1),
    shift_labels.view(-1)               # Flatten to (batch*(T-1),)
)  # Scalar — this is what we backpropagate!

# --- INFERENCE: Generate next token ---
next_logits = logits[:, -1, :]            # (batch, vocab_size)
scaled = next_logits / 0.7               # Temperature scaling
probs = F.softmax(scaled, dim=-1)         # Probability distributio
next_token = torch.multinomial(probs, 1)  # Sample one token
```

Part 5: Loss Functions and Metrics

29

The Training Objective: Next-Token Prediction

- The entire LLM training objective is deceptively simple:
- **Given a sequence of tokens, predict the next token at every position**
- Input: ["The", "cat", "sat", "on", "the"] → Target: ["cat", "sat", "on", "the", "mat"]
- The model outputs probability distributions; the training data provides the correct answers
- This single objective, scaled across **trillions of tokens**, produces general intelligence
- No task-specific labels needed — the text itself IS the supervision signal
- GPT-5.2, Claude Opus 4.6, Gemini 3 Pro, DeepSeek V3.2, Kimi K2.5 — all trained this way
- GPT-OSS uses this exact same objective

30

Cross-Entropy Loss

- **Cross-entropy** measures how far the model's predicted distribution is from the true answer
- True distribution: a **one-hot vector** — all probability mass on the correct next token
- Loss = −log P(correct_token) — the negative log-probability of the right answer
- If model assigns P = 0.9 to correct token → loss = −log(0.9) = 0.105 (low → good!)
- If model assigns P = 0.01 to correct token → loss = −log(0.01) = 4.605 (high → bad!)
- Computed at **every position** simultaneously, averaged across the sequence and batch
- Cross-entropy next-token prediction is the primary objective
- Some modern models add auxiliary objectives (e.g., multi-token prediction) or architecture-specific training tricks

31

Teacher Forcing: Why Training Is So Efficient

- **Problem**: If we generated one token at a time during training, it would be purely sequential and impossibly slow
- **Teacher forcing**: Feed the entire correct sequence as input, predict all next tokens **in parallel**
- The causal mask (triangular mask in attention) ensures each position can only see itself and earlier positions
- So every position effectively sees a different-length prefix — but all computed in **ONE forward pass**:
- Position 0 sees ["The"] → predicts "cat"
- Position 1 sees ["The", "cat"] → predicts "sat"
- Position 2 sees ["The", "cat", "sat"] → predicts "on" — all computed simultaneously!
- This is why transformer training is massively parallelizable — and why GPUs changed everything

32

Perplexity — The Key LLM Metric

- **Perplexity (PPL)** = exp(average cross-entropy loss) — the standard language model metric
- Intuition: "How many tokens is the model effectively choosing between at each step?"
- **PPL = 1**: Perfect prediction — model assigns probability 1.0 to every correct token (impossible in practice)
- **PPL = 100**: Model is as uncertain as choosing uniformly among 100 options
- **PPL = vocab_size**: Model is guessing randomly — worst case
- **Lower perplexity = better model** — this is the primary metric during pre-training
- On classic LM benchmarks, large frontier models often reach single-digit to low-teens perplexities

33

Other Metrics You'll See

- **Training loss**: Raw cross-entropy plotted over training steps — should decrease smoothly
- **Validation loss**: Cross-entropy on held-out data — if it starts increasing while training loss decreases → overfitting, stop!
- **Bits-per-byte (BPB)**: Cross-entropy normalized by byte count (not token count) — allows fair comparison across different tokenizers
- **Token accuracy**: Fraction of positions where argmax = correct token — intuitive but coarse
- **Downstream benchmarks** (after training): MMLU, HumanEval, MATH, GPQA, ARC, Chatbot Arena Elo

34

Computing Loss and Perplexity

PYTHON

```
import torch
import torch.nn.functional as F
import math

def compute_loss_and_perplexity(logits, labels):
    """Standard language modeling loss computation."""
    # logits: (batch, seq_len, vocab_size)
    # labels: (batch, seq_len) — the true next tokens
    loss = F.cross_entropy(
        logits.view(-1, logits.size(-1)),  # Flatten: (batch*seq_le
        labels.view(-1),                    # Flatten: (batch*seq_l
        ignore_index=-100                   # Ignore padding positi
    )
    perplexity = math.exp(loss.item())      # PPL = e^(cross-entrop
    return loss, perplexity

# Example: inside the training loop
for batch in dataloader:
    token_ids = batch["input_ids"]                 # (batch, seq_le
    logits = model(token_ids[:, :-1])               # Predict from
    labels = token_ids[:, 1:]                       # True next tok
    loss, ppl = compute_loss_and_perplexity(logits, labels)
    print(f"Loss: {loss.item():.4f}, Perplexity: {ppl:.2f}")
    loss.backward()                                 # Backpropagate
    optimizer.step()
    optimizer.zero_grad()
```

35

Part 6: The Full Architecture — End to End

36

Why Decoder-Only?

- The 2017 Transformer had both **encoder** and **decoder** stacks
- Modern LLMs use **only the decoder stack** with causal masking — why?
- **Simpler**: One architecture, one objective (next-token prediction) — nothing else needed
- **Scales predictably**: Bigger model + more data = better results (scaling laws)
- **Versatile**: Generation subsumes understanding — classify by generating the answer
- GPT-5.2, Claude Opus 4.6, Gemini 3 Pro, DeepSeek V3.2, Qwen3, Kimi K2.5, MiniMax M2.1 — **all decoder-only**
- Encoder-only models (e.g., for retrieval/embedding) still exist in niche roles, but decoder-only dominates
- GPT-OSS is decoder-only — this is the architecture we'll study later

🚀 **Interactive Demo:** bert_vs_gpt_demo.html

37

End-to-End Walkthrough

- **Input**: "The cat sat on" (raw string)
- **Tokenize**: ["The", " cat", " sat", " on"] → [464, 3797, 3332, 319] — 4 integer IDs
- **Embed**: Look up 4 token vectors + add 4 positional vectors → 4 vectors of dim d_model
- **Transform**: Pass through N transformer layers (attention + FFN) → 4 contextualized vectors
- **Project**: LM head produces logits of shape (4, vocab_size) — one distribution per position
- **Train**: Cross-entropy loss between predicted distributions and actual next tokens ["cat", " sat", " on", " the"]
- **Generate**: Take last position's logits → temperature → top-p → sample → decode → append → repeat
- This is the **complete GPT pipeline**

38

Simplified GPT — The Skeleton You'll Expand Into GPT-OSS

PYTHON

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class GPTModel(nn.Module):
    def __init__(self, vocab_size=200000, max_seq_len=4096,
                 embed_dim=4096, n_layers=32, n_heads=32):
        super().__init__()
        self.token_embed = nn.Embedding(vocab_size, embed_dim)
        self.pos_embed = nn.Embedding(max_seq_len, embed_dim)  # Wi
        self.dropout = nn.Dropout(0.1)
        self.layers = nn.ModuleList([
            TransformerBlock(embed_dim, n_heads)  # From last lectu
            for _ in range(n_layers)
        ])
        self.norm = nn.LayerNorm(embed_dim)                    # Wi
        self.lm_head = nn.Linear(embed_dim, vocab_size, bias=False)
        self.lm_head.weight = self.token_embed.weight          # We

    def forward(self, token_ids, targets=None):
        B, T = token_ids.shape
        tok_emb = self.token_embed(token_ids)
        pos_emb = self.pos_embed(torch.arange(T, device=token_ids.d
        x = self.dropout(tok_emb + pos_emb)
        for layer in self.layers:
            x = layer(x)             # N transformer blocks
        x = self.norm(x)             # Final normalization
        logits = self.lm_head(x)     # (B, T, vocab_size)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)), targets.view(-1)
            )
        return logits, loss
```

What Changes in the Original Transformer

- The code above uses the **2017 original Transformer** components you already know
- Modern frontier models have the following changes:
- • **nn.LayerNorm** → **RMSNorm** (simpler, faster — used by GPT-OSS, DeepSeek V3.2, Qwen3)
- • **Learned positional embeddings** → **RoPE** (rotary — supports longer contexts, better extrapolation)
- • **Standard multi-head attention** → **GQA** (Grouped Query Attention — memory efficient at inference)
- • **ReLU in FFN** → **SwiGLU** (smoother activation, consistently better performance)
- • **Post-norm** → **Pre-norm** (layer norm before attention/FFN — more stable training)
- • + **KV-cache** and **flash attention** for efficient inference and training
- These are **drop-in replacements** — the overall architecture and pipeline stay exactly the same

40

Part 7: HuggingFace in Practice

41

HuggingFace — Your Practical Toolkit

- 🤗 **Transformers**: Unified API for 500K+ pre-trained models — the industry standard
- 🤗 **Tokenizers**: Fast, Rust-backed tokenization — train BPE tokenizers in minutes
- 🤗 **Datasets**: Large collection of pre-processed training and evaluation datasets
- **AutoTokenizer**: Automatically loads the correct tokenizer for any model on the Hub
- **AutoModelForCausalLM**: Loads any decoder-only language model with one line
- HuggingFace is used extensively for:
- Training the tokenizer, loading/saving checkpoints, data preprocessing, evaluation

42

End-to-End: Tokenize → Forward → Inspect → Generate

PYTHON

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

# --- Step 1: Tokenize ---
text = "The future of AI is"
inputs = tokenizer(text, return_tensors="pt")
print(f"Tokens: {tokenizer.tokenize(text)}")  # ['The', 'Ġfuture',
print(f"IDs:    {inputs.input_ids}")           # tensor([[464, 2003

# --- Step 2: Forward pass → inspect logits ---
with torch.no_grad():
    logits = model(**inputs).logits            # (1, 5, 50257)

# See what the model predicts after each token
for i in range(logits.shape[1]):
    prefix = tokenizer.decode(inputs.input_ids[0, :i+1])
    top_pred = tokenizer.decode(logits[0, i].argmax())
    print(f"  After '{prefix}' → predicts '{top_pred}'")

# --- Step 3: Generate ---
out = model.generate(inputs.input_ids, max_new_tokens=30, temperatu
                     do_sample=True)
print(f"\nGenerated: {tokenizer.decode(out[0])}")
```

43

Training a BPE Tokenizer from Scratch

PYTHON

```
from tokenizers import Tokenizer, models, trainers, pre_tokenizers

# Step 1 of building GPT: train your own tokenizer
tokenizer = Tokenizer(models.BPE())
tokenizer.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space

trainer = trainers.BpeTrainer(
    vocab_size=50000,                       # Target vocabulary siz
    special_tokens=["<|endoftext|>",        # End of sequence
                    "<|pad|>",              # Padding
                    "<|begin_of_text|>"],   # Beginning of sequence
    min_frequency=2,                         # Minimum pair frequen
)

# Train on your corpus — this learns the merge rules
tokenizer.train(files=["corpus_part1.txt", "corpus_part2.txt"], tra
tokenizer.save("gpt-oss-tokenizer.json")

# Test it
output = tokenizer.encode("Hello, GPT-OSS!")
print(f"Tokens: {output.tokens}")
print(f"IDs:    {output.ids}")
# This tokenizer is trained ONCE, then used for ALL model training
```

Summary

45

What You Now Understand

- **Tokenization**: Raw text → subword tokens → integer IDs via byte-level BPE
- **Vocabulary**: A fixed set of tokens (100K–200K) trained on corpus data before model training
- **Special tokens**: Structural markers (<|endoftext|>, <|pad|>) added to the vocabulary manually
- **Tokenization artifacts**: Why LLMs struggle with arithmetic, spelling, and multilingual equality
- **Token embeddings (W_E)**: Learnable lookup table mapping IDs → dense vectors
- **Positional embeddings**: Injecting sequence order (learned or RoPE — next lecture)
- **Weight tying**: Reusing W_E as the output projection — fewer parameters, better performance
- **LM Head** → **Logits** → **Softmax** → **Sampling**: The complete output pipeline
- **Cross-entropy loss**: −log P(correct next token) — the sole pre-training objective
- **Teacher forcing**: Predict all next tokens in parallel using the causal mask
- **Perplexity**: exp(loss) — "how many tokens is the model choosing between?" — lower is better

46

The Road to GPT-OSS

- ✅ **1**: Original Transformer architecture (2017) — attention, FFN, residuals, norms
- ✅ **2**: NLP fundamentals — tokenization, embeddings, loss, metrics, output pipeline
- 📍 **3**: Modern transformer upgrades — RoPE, RMSNorm, SwiGLU, GQA, KV-cache, flash attention
- 📍 **4**: GPT-OSS from scratch — data pipeline, tokenizer training, model implementation
- 📍 **5**: Pre-training at scale — distributed training, scaling laws, training dynamics
- 📍 **6**: Post-training — SFT, RLHF/DPO, alignment, from base model to assistant

47

Interactive Demos

- 📊 **Tokenization:**

  🚀 **Interactive Demo:** bpe_algorithm_demo.html

  🚀 **Interactive Demo:** tokenizer_demo.html

- 📐 **Embeddings & Architectures:**

  🚀 **Interactive Demo:** embeddings_demo.html

  🚀 **Interactive Demo:** bert_vs_gpt_demo.html

48

Supplementary Resources

🚀 **Interactive Demo:** handout.html

49